

Relatório de Auditoria de Segurança — Assídua

Isolamento de tenant, permissões, IDOR, segredos e inputs sem tratamento

DATA

2026-09-08

REPOSITÓRIO

github.com/rilsonjoas/assidua

ESCOPO AUDITADO

Monorepo assidua (api/ Laravel 13 + Sanctum + Policies, app/ Expo/React Native). Escopo: rotas autenticadas em routes/api.php, Controllers, Policies, Models, exportação LGPD, webhook RevenueCat, fluxo de autenticação (magic link + Google OAuth) e superfície de renderização do app mobile.

NOTA METODOLÓGICA

Mecanismo de isolamento de tenant detectado: Policies do Laravel (Gate::authorize) contra Profile::isAccessibleBy() — dono OU colaborador/cuidador aceito — equivalente funcional ao RLS do Supabase citado na metodologia padrão. “Permissão no navegador” mapeada para o gate is_owner do app RN comparado linha a linha com os métodos update()/delete() de cada Policy. XSS mapeado para a superfície real do RN (sem WebView/HTML) mais a exportação CSV, onde o equivalente de “input sem tratamento” é CSV Formula Injection.

3 achados — 1 Alta · 1 Baixa · 1 Informativa

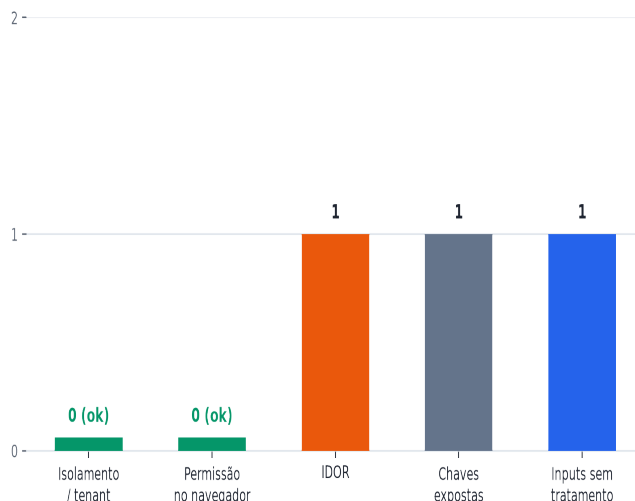
Resumo executivo

3 achados verificados em código real — nenhum especulativo. 1 de severidade Alta (IDOR real, com caminho de exploração concreto), os outros dois de baixo risco/hardening. As categorias 1 (isolamento de tenant) e 2 (permissão definida no navegador) não tiveram achados: verificação sistemática, rota por rota, confirmou cobertura correta.

Achados por severidade



Achados por categoria (verde = nenhum achado, categoria coberta)



Pontos fortes (verificados, com evidência)

✓ **Isolamento de tenant (Policies)** — Todo Controller (Profile, Medication, DoseSchedule, DoseLog, Stock) chama Gate::authorize() ANTES de tocar o model, contra Profile::isAccessibleBy() (dono OU colaborador aceito) — verificado rota por rota em routes/api.php.

✓ **Paridade UI (frontend) × Policy (backend)** — O gate is_owner que o app RN usa pra esconder editar/apagar bate 1:1 com ProfilePolicy::update/delete e MedicationPolicy::update/delete (owner-only) — nenhuma ação exclusiva de dono depende só da UI escondida.

✓ **Sem segredo hardcoded** — Toda credencial (Google, Resend, AWS, RevenueCat, Sentry) vem de env(), sem valor-default inseguro; webhook do RevenueCat falha fechado (401) quando o secret está vazio.

✓ **Fluxo de auth robusto** — Magic link usa token de 64 bytes hashado (sha256) no banco, expira em 15 min, invalida o anterior a cada novo pedido. OAuth Google valida state contra allowlist de origem + path /auth-callback, nunca devolve token em JSON aberto sem destino validado.

✓ **Exportação LGPD via signed URL** — GET /me/export usa assinatura do Laravel com expiração de 10 min e o id do usuário embutido na assinatura — trocar o id na query invalida a URL antes de chegar no controller.

✓ **Mass assignment sob controle** — subscription_tier é fillable no model User, mas o único write path é RevenueCatWebhookController com array literal explícito — nenhuma rota deixa o próprio usuário setar o campo via request bruto.

Pontos fracos (risco central)

O IDOR de **POST /dose-logs** (A1) é o risco real deste levantamento: permite ler dado de medicamento de outro perfil e corromper registros de adesão de terceiros, sem nunca passar pela Policy do perfil-alvo. Os outros dois achados são hardening de baixo risco, sem exploração cross-tenant demonstrada hoje.

Achados detalhados

Alta

A1 · IDOR

api/app/Http/Controllers/DoseLogController.php:
143-180

IDOR em POST /dose-logs — dose_schedule_id/medication_id não são validados contra o profile autorizado

O endpoint recebe profile_id, dose_schedule_id e medication_id no body. Gate::authorize('create', [DoseLog::class, \$profile]) checa só que o profile_id pertence (ou é compartilhado com) o usuário — mas dose_schedule_id e medication_id são validados apenas com 'exists:tabela,id' (existem em QUALQUER lugar do banco, não no profile autorizado). Como o updateOrCreate casa por dose_schedule_id + scheduled_at, e os IDs são autoincremento sequencial

(database/migrations/2026_06_28_000005_create_dose_logs_table.php:12), um usuário autenticado qualquer pode enviar um dose_schedule_id de OUTRO perfil junto com o próprio profile_id (que passa no Gate) e: (1) ler nome/dosagem do medicamento alheio na resposta; (2) sobrescrever um DoseLog existente de outro perfil — status, notes, e o próprio profile_id do registro.

```
$data = $request->validate([
    'dose_schedule_id' => 'required|exists:dose_schedules,id',
    'medication_id' => 'required|exists:medications,id',
    'profile_id' => 'required|exists:profiles,id',
    ...
]);
$profile = Profile::findOrFail($data['profile_id']);
Gate::authorize('create', [DoseLog::class, $profile]); // só valida profile_id
$log = DoseLog::updateOrCreate(
    ['dose_schedule_id' => $data['dose_schedule_id'], 'scheduled_at' => ...],
    array_merge($data, [...]) // profile_id, medication_id do body, não checados
);
```

Impacto: Vazamento de nome/dosagem de medicamento de outro paciente e corrupção de registros de adesão (status/notas) de terceiros — dado que pode chegar a um resumo de consulta médica (GenerateConsultationSummary) sem nunca ter passado pela Policy do perfil-alvo.

Correção sugerida: Resolver dose_schedule_id e medication_id a partir do próprio \$profile autorizado (ex.: \$profile->medications()->findOrFail(...) e whereHas('medication', fn(\$q) => \$q->where('profile_id', \$profile->id))), em vez de aceitar os três IDs soltos e independentes do body.

Baixa

A2 · Inputs sem tratamento (equiv. XSS)

api/app/Http/Controllers/DataExportController.php:~98-149

Exportação CSV (LGPD) sem neutralização de CSV Formula Injection

downloadCsv() escreve nome/dosagem/instruções/notas do medicamento direto em células CSV sem prefixar valores que começam com =, +, -, ou @. Se um desses campos contiver uma fórmula, o Excel/Sheets pode executá-la ao abrir o arquivo. XSS tradicional não se aplica (app RN sem WebView/HTML), este é o equivalente real na stack. Severidade rebaixada porque todo campo exportado só pode ter sido escrito pelo próprio dono do perfil (MedicationController exige Policy 'update', owner-only) — não há atacante cruzando fronteira de usuário aqui, é auto-injeção na própria exportação.

```
fputcsv($handle, [
    $profile->name, $medication->name, $medication->dosage ?? '',
    $medication->unit ?? '', $medication->instructions ?? '',
    $medication->notes ?? '', ...
], ';'); // sem sanitização contra =/+/-/@ no início da célula
```

Impacto: Execução de fórmula no Excel/Sheets do próprio usuário ao abrir seu export. Sem cruzamento de tenant — impacto limitado a auto-exploração.

Correção sugerida: Prefixar apóstrofo (') em qualquer célula cujo valor comece com =, +, -, @, tab ou CR antes do fputcsv — padrão OWASP para CSV Injection.

Informativa

A3 · Chaves expostas / segredos

api/app/Http/Controllers/RevenueCatWebhookController.php:47

Comparação do webhook secret não é constant-time

\$request->header('Authorization') !== \$secret usa comparação padrão de string do PHP, que retorna assim que encontra o primeiro byte diferente — abre uma janela teórica de timing side-channel contra o valor do secret. Nenhum hardcode encontrado no projeto (achado positivo, ver seção de pontos fortes); este item é só sobre a FORMA da comparação, não sobre o segredo estar exposto.

```
if (! $secret || $request->header('Authorization') !== $secret) {
    return response()->json(['error' => 'unauthorized'], 401);
}
```

Impacto: Baixo risco prático (webhook de um único fornecedor, volume baixo de tentativas), mas é boa prática padrão trocar para comparação constant-time em qualquer comparação de segredo.

Correção sugerida: Trocar !== por hash_equals(\$secret, (string) \$request->header('Authorization')).

Recomendações priorizadas

Prior.	Ação	Por quê
P1	Corrigir o IDOR de POST /dose-logs (A1)	Resolver dose_schedule_id/medication_id a partir do profile já autorizado, não como IDs soltos do body. É o único achado com impacto real de confidencialidade/integridade entre usuários.
P2	Sanitizar exportação CSV contra Formula Injection (A2)	Prefixar apóstrofo em células que começam com =, +, -, @ antes de escrever no CSV. Baixo esforço, fecha o gap mesmo sem atacante cross-tenant hoje.
P3	Trocar comparação do webhook secret para hash_equals() (A3)	Troca de uma linha, remove a janela teórica de timing attack.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar em cada issue.

--- ISSUE 1 ---

[Segurança] IDOR em POST /dose-logs permite ler e sobrescrever registros de outro perfil

Labels: security, severidade:alta

****Problema****

``DoseLogController::store()`` (``api/app/Http/Controllers/DoseLogController.php:143-180``, rota ``POST /dose-logs``) recebe ``profile_id``, ``dose_schedule_id`` e ``medication_id`` no body. ``Gate::authorize('create', [DoseLog::class, $profile])`` valida só o ``profile_id``. ``dose_schedule_id``/``medication_id`` são validados apenas com ``exists:tabela,id`` – existem em QUALQUER lugar do banco, não necessariamente no profile autorizado.

Como os IDs são autoincremento sequencial

(``database/migrations/2026_06_28_000005_create_dose_logs_table.php:12``) e o ``updateOrCreate`` casa por ``dose_schedule_id`` + ``scheduled_at``, um usuário autenticado qualquer pode enviar seu próprio ``profile_id`` (passa no Gate) junto com um ``dose_schedule_id`` de OUTRO perfil.

****Por que é explorável****

Nenhuma verificação liga ``dose_schedule_id``/``medication_id`` ao ``profile_id`` autorizado – as três IDs são tratadas como independentes.

****Evidência****

```
```php
$data = $request->validate([
 'dose_schedule_id' => 'required|exists:dose_schedules,id',
 'medication_id' => 'required|exists:medications,id',
 'profile_id' => 'required|exists:profiles,id',
]);
$profile = Profile::findOrFail($data['profile_id']);
Gate::authorize('create', [DoseLog::class, $profile]);
$log = DoseLog::updateOrCreate(
 ['dose_schedule_id' => $data['dose_schedule_id'], 'scheduled_at' => ...],
 array_merge($data, [...])
);
```
```

****Impacto****

- Leitura de nome/dosagem de medicamento de outro perfil na resposta.
- Sobrescrita de status/notas de um ``DoseLog`` de outro perfil, incluindo reatribuição do ``profile_id`` do registro.
- Dado pode chegar a ``GenerateConsultationSummary`` (resumo pra consulta médica).

****Sugestão de correção****

Resolver ``dose_schedule_id``/``medication_id`` a partir do ``$profile`` já autorizado (ex.: ``$profile->medications()->findOrFail($data['medication_id'])`` e uma busca de schedule escapada por ``whereHas('medication', fn($q) => $q->where('profile_id', $profile->id))``), em vez de aceitar os três IDs soltos do body.

****Critérios de aceite****

- [] ``dose_schedule_id`` enviado no body é rejeitado (404/422) se não pertencer ao ``profile_id`` autorizado.
- [] ``medication_id`` enviado no body é rejeitado se não pertencer ao mesmo profile.
- [] Teste automatizado cobrindo: usuário A autenticado tenta criar/sobrescrever um ``DoseLog`` usando

`dose\_schedule\_id` do perfil do usuário B → 403/404, sem vaziar dado do medicamento de B na resposta.
- [] Regressão: fluxo normal (dono ou colaborador aceito registrando dose do próprio perfil) continua funcionando sem mudança de contrato pro app.
--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Hardening: CSV Formula Injection na exportação LGPD e comparação não constant-time do webhook secret

Labels: security, severidade:baixa

Dois achados de baixo risco agrupados por serem hardening pontual, sem impacto cross-tenant hoje.

****1. CSV Formula Injection – `api/app/Http/Controllers/DataExportController.php` (~linhas 98-149)****

`downloadCsv()` escreve nome/dosagem/instruções/notas do medicamento direto em células CSV sem neutralizar valores que começam com `=`, `+`, `-` ou `@`. Se um desses campos contiver uma fórmula, Excel/Sheets pode executá-la ao abrir o export. Impacto limitado: todo campo exportado só pode ter sido escrito pelo próprio dono do perfil (`MedicationController` exige Policy `update`, owner-only), então é auto-injeção – mas vale corrigir como defesa em profundidade.

***Correção sugerida*:** prefixar apóstrofo (`'`) em qualquer célula cujo valor comece com `=`, `+`, `-`, `@`, tab ou CR antes do `fputcsv`.

****2. Comparação não constant-time – `api/app/Http/Controllers/RevenueCatWebhookController.php:47`****

```
```php
if (! $secret || $request->header('Authorization') !== $secret) {
```
```

`!==` retorna assim que encontra o primeiro byte diferente – janela teórica de timing side-channel contra o valor do secret. Nenhum segredo hardcoded foi encontrado no projeto; é só a forma da comparação.

***Correção sugerida*:** `hash\_equals(\$secret, (string) \$request->header('Authorization'))`.

****Critérios de aceite****

- [] Export CSV: valor de teste começando com `=` some do arquivo como fórmula ativa (aparece como texto literal, com apóstrofo ou aspas).
- [] `RevenueCatWebhookController::handle` usa `hash\_equals()`.
- [] Teste automatizado do webhook continua passando com secret correto/incorreto.

--- FIM ISSUE 2 ---
